

Reviewer Pack — Minimal Local System Rank and Resolution Proof Width Lower B...

Entry a003094983b8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 00:17:03 UTC

Minimal Local System Rank and Resolution Proof Width Lower Bound

Entry ID: a003094983b8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 00:17:03 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every 3CNF formula φ , the minimal local system rank ($\text{mls}(\varphi)$) of its associated matrix representation is lower bounded by a function of its resolution proof width, i.e., $\text{mls}(\varphi) = \Omega(w(\varphi)^2)$, where $w(\varphi)$ is the resolution proof width.

Rationale (proposer's reasoning):

Algebraic geometry provides tools to study the geometric properties of complex objects. Local system rank measures the complexity of certain algebraic structures, which could provide insights into the complexity of proof systems like resolution. If a direct relationship between local system rank and proof width exists, it suggests a deeper connection between geometric and computational complexities.

Taxonomy category: Algebraic Geometry × Resolution Proof Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7e2d81e5504bee9f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated 3CNF formulas φ with n variables ($n \leq 40$), the minimal local system rank $\text{mls}(\varphi)$ is within $\pm 10\%$ of $w(\varphi)^2$ for at least 80% of the seeds. The conjecture is falsified if any seed produces a deviation greater than 110% from $w(\varphi)^2$ or if any seed results in $\text{mls}(\varphi) > 1.1 * w(\varphi)^2$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local system rank" AND "resolution proof complexity" - "algebraic geometry" AND "resolution proof width lower bound" - "matrix representation" AND "proof width in algebraic geometry"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_3cnf(n: int) -> list:
    clauses = []
    for _ in range(2 * n):
        literals = [random.choice([1, -1]) * (i + 1) for i in range(n)]
        clause = random.sample(literals, 3)
        clauses.append(clause)
    return clauses

def matrix_representation(clauses: list, n: int) -> list:
    m = len(clauses)
    A = [[0] * n for _ in range(m)]
    for i, clause in enumerate(clauses):
        for literal in clause:
            var_index = abs(literal) - 1
            if literal > 0:
                A[i][var_index] = 1
            else:
                A[i][var_index] = -1
    return A

def gaussian_elimination(A: list, n: int) -> list:
    m = len(A)
    for i in range(n):
        # Find pivot
        pivot_row = next((j for j in range(i, m) if A[j][i] != 0), None)
        if pivot_row is None:
            continue
```

```

A[i], A[pivot_row] = A[pivot_row], A[i]

# Eliminate below pivot
for j in range(i + 1, m):
    factor = A[j][i] / A[i][i]
    for k in range(n):
        A[j][k] -= factor * A[i][k]

return A

def minimal_local_system_rank(A: list) -> int:
    n = len(A[0])
    rank = 0
    for i in range(n):
        if any(A[j][i] != 0 for j in range(len(A))):
            rank += 1
    return rank

def resolution_proof_width(clauses: list) -> int:
    # Simplified DPLL solver to estimate proof width
    def solve(model, clause):
        for literal in clause:
            if (literal > 0 and literal in model) or (literal < 0 and -literal not in model):
                return True
        return False

    def dpll(clauses, model):
        unit_clauses = [c for c in clauses if len(c) == 1]
        while unit_clauses:
            literal = unit_clauses.pop()
            model.add(literal)
            new_clauses = []
            for clause in clauses:
                if literal in clause:
                    continue
                if -literal in clause:
                    new_clauses.append([l for l in clause if l != -literal])
                else:
                    new_clauses.append(clause)
            unit_clauses.extend([c for c in new_clauses if len(c) == 1])
            clauses = new_clauses

        for literal in set(lit for clause in clauses for lit in clause):
            if not solve(model, [literal]):
                return dpll(clauses, model | {literal})
        return model

    proof_width = 0
    for i in range(len(clauses)):
        model = set()
        if not dpll(clauses[:i] + clauses[i+1:], model):
            proof_width += 1
    return proof_width

def run_trial(seed: int) -> dict:
    random.seed(seed)

```

```

n = random.choice([5, 10, 15, 20, 30, 40])
formula = generate_3cnf(n)
A = matrix_representation(formula, n)
mls_phi = minimal_local_system_rank(A)
w_phi = resolution_proof_width(formula)

if mls_phi > 1.1 * w_phi**2:
    return {
        "metric_name": "mls( $\phi$ )",
        "metric_value": mls_phi,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": f"mls( $\phi$ ) > 1.1 * w( $\phi$ )^2 for n={n}"
    }

return {
    "metric_name": "mls( $\phi$ )",
    "metric_value": mls_phi,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ranks = [r["metric_value"] for r in results if r["conjecture_holds"]]
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(mean_ranks) / len(mean_ranks):.2f} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mls( $\phi$ ) > 1.1 * w( $\phi$ )^2\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unreachable")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c9c05bd9.py", line 133, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c9c05bd9.py", line 106, in run_trial
    w_phi = resolution_proof_width(formula)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c9c05bd9.py", line 95, in resolution_pr
    if not dpll(clauses[:i] + clauses[i+1:], model):
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c9c05bd9.py", line 89, in dpll
    return dpll(clauses, model | {literal})
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c9c05bd9.py", line 89, in dpll
    return dpll(clauses, model | {literal})
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c9c05bd9.py", line 89, in dpll
    return dpll(clauses, model | {literal})
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
[Previous line repeated 993 more times]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c9c05bd9.py", line 87, in dpll
    for literal in set(lit for clause in clauses for lit in clause):
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
RecursionError: maximum recursion depth exceeded

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execu-
tion to verify the conjecture. | next: Re-run the test with increased recursion limits or use
a different algorithm that does not exceed maximum recursion depth.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22776
2	preregistration	ollama_remote	glm4:latest	0	0	10171
3	novelty	ollama_remote	glm4:latest	0	0	8283
4	novelty	ollama_remote	glm4:latest	0	0	13420
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22857
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22797
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14894
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22926
9	judge	ollama_remote	glm4:latest	0	0	13580

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 151705 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/a003094983b8.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/a003094983b8.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/a003094983b8.tar.gz (if generated)